arXiv:2411.00774v5 [cs.SD] 8 Dec 2024

Freeze-Omni: A Smart and Low Latency Speech-to-speech Dialogue Model with Frozen LLM

Xiong Wang[1,*], Yangze Li[2], Chaoyou Fu[3], Yunhang Shen[1]
Lei Xie[2], Ke Li[1], Xing Sun[1], Long Ma[1,†]

[1]Tencent Youtu Lab
[2]Audio, Speech and Language Processing Group (ASLP@NPU)
[3]Nanjing University
[*] Main Contribution [†] Corresponding Author

https://freeze-omni.github.io/

Abstract

Rapidly developing large language models (LLMs) have brought tremendous intelligent applications. Especially, the GPT-4o's excellent duplex speech interaction ability has brought impressive experience to users. Researchers have recently proposed several multi-modal LLMs in this direction that can achieve user-agent speech-to-speech conversations. This paper proposes a novel speech-text multimodal LLM architecture called **Freeze-Omni**. Our main contribution is that the speech input and output modalities can be easily connected to a textual LLM while keeping the LLM's parameters frozen throughout the training process. We design a three-stage training strategy for modeling both the speech input and output, enabling Freeze-Omni to obtain speech-to-speech conversation ability using text-speech paired data (such as ASR and TTS data) and **only 60,000 multi-round text Q&A data on 8 GPUs**. Moreover, we can effectively ensure that the intelligence of the Freeze-Omni in the speech modality is at the **same level** compared with that in the text modality of its backbone LLM, while achieving low latency end-to-end spoken response. In addition, we also designed a method to achieve duplex dialogue ability through multi-task training, giving Freeze-Omni a more natural style of dialogue ability between users and agents. In summary, Freeze-Omni holds great potential to conduct speech-to-speech dialogue based on a multimodal LLM under the condition of a frozen LLM, avoiding the catastrophic forgetting problem caused by limited data and training resources.

1 Introduction

Recent years have witnessed rapid development of large language models (LLMs). The family of LLMs represented by the GPT series [10, 1] of OpenAI has demonstrated extraordinary capabilities. As speech interaction is one of the most natural forms of human-computer interaction, combining speech input and output with an LLM can bring a natural experience to users. The traditional method is to use a cascaded approach of ASR + LLM + TTS to achieve the interaction with LLM in speech modality. However, this engineering-centered pipeline approach often leads to a considerable interaction latency. Nevertheless, GPT-4o [18] has changed this situation – it provides an end-to-end speech interaction mode which has significantly improved user experience, triggering a research boom regarding multimodal LLMs for speech-to-speech interaction.

In the field of general LLMs, many public models such as Llama 3.2 [8], Qwen2.5 [21], Mixtral [14], etc., have provided good opportunities for researchers to develop downstream tasks. Therefore, in the field of multimodal LLMs targeting speech-to-speech conversation, works such as Mini-Omni2 [24], LLaMA-Omni [9], and Moshi [7] have provided excellent references for researchers. These works

Email: wangxiongts@gmail.com, malonema@tencent.com

adopt different strategies to align the speech modality with an LLM and design some tricks to achieve a duplex dialogue mode.

In this research context, we found that in the process of aligning the LLM with the speech modality in existing public speech-text multimodal LLMs [6, 7, 9, 11, 28, 23], the parameters of the LLM are more or less fine-tuned. However, in most cases, it is very difficult for researchers to easily collect spoken Q&A data at the million-hour level (the corresponding text content can be comparable to the amount of data for training text-modal LLMs). Fine-tuing the LLM inevitably brings about the catastrophic forgetting problem to the LLM, resulting in a negative impact on its original "intelligence" ability. In addition, only a few works have evaluated the perfomrance of spoken question-answering tasks for speech-to-speech multimodal LLMs, showing an obvious gap in the performance between spoken Q&A and text-modality Q&A. Therefore, in this paper, we propose a speech-to-speech dialogue LLM called *Freeze-Omni*, which achieves effective speech-text modality alignment while keeping the LLM parameters frozen, obtaining low latency speech dialogue capabilities while maintaining the orignal intelligence of the backbone LLM. Specifically, Freeze-Omni is mainly implemented in the following steps:

Modeling of speech input We first use a large amount of ASR data to align the speech encoder and the LLM, enabling the LLM to understand the semantic information from the speech. Then, with the LLM frozen, a training strategy of prompt embedding is used to let the model have the ability to possess speech input to text output, training on only a small amount of Q&A data.

Modeling of speech output Second, we use a sizable amount of text-speech paired data to train the AR-based speech decoder which can generate speech tokens from text and a single-codebook based codec model is used to decode the speech token into waveform. Then, we design a prefix kv-cache fine-tune strategy, using the hidden state vector output by the LLM to transfer the speech decoder into the output text space of LLM, achieving the ability of text input to speech output while keeping the LLM frozen.

Design for duplex dialogue Finally, we connect the speech encoder and speech decoder from the above parts to the backbone LLM. Then, a task of chunk-wise state prediction is used to determine whether or not the user interrupts the dialogue, achieving the duplex speech-to-speech dialogue ability.

In conclusion, the main contributions of Freeze-Omni are as follows:

- The parameters of the LLM are completely frozen throughout the training process of Freeze-Omni, maintaining the original intelligence of the LLM and achieving low latency speech-to-speech dialogue at the same time.
- The paired text-speech Q&A training data is at a small scale and consumes fewer computing resources in the building of Freeze-Omni.
- Freeze-Omni can support any (multimodal) LLM that has a text modality and retains the abilities of the LLM such as prompt following and role-playing. Moreover, if it is necessary to change the style of the LLM's response, it is only necessary to fine-tune the LLM with text data in the corresponding style.

2 Model

2.1 Overview

Freeze-Omni is a speech-to-speech dialogue model and the architecture is shown in Fig. 1, exhibiting the characteristic of being "smart" as it is constructed upon a "frozen" text-modality LLM. This enables it to keep the original intelligence of the LLM backbone, without being affected by the forgetting problem induced by the fine-tuning process for integration of the speech modality. Specifically, Freeze-Omni contains a speech encoder that supports streaming speech input and a speech decoder that generates streaming output speech. During the training process, Freeze-Omni first achieves the alignment between speech input to text output, and then the text input to speech output. Finally, by connecting these two components with the LLM, the ability of speech input to speech output is obtained. This section will provide a detailed introduction to the architecture, training strategy, and duplex dialogue design of Freeze-Omni.

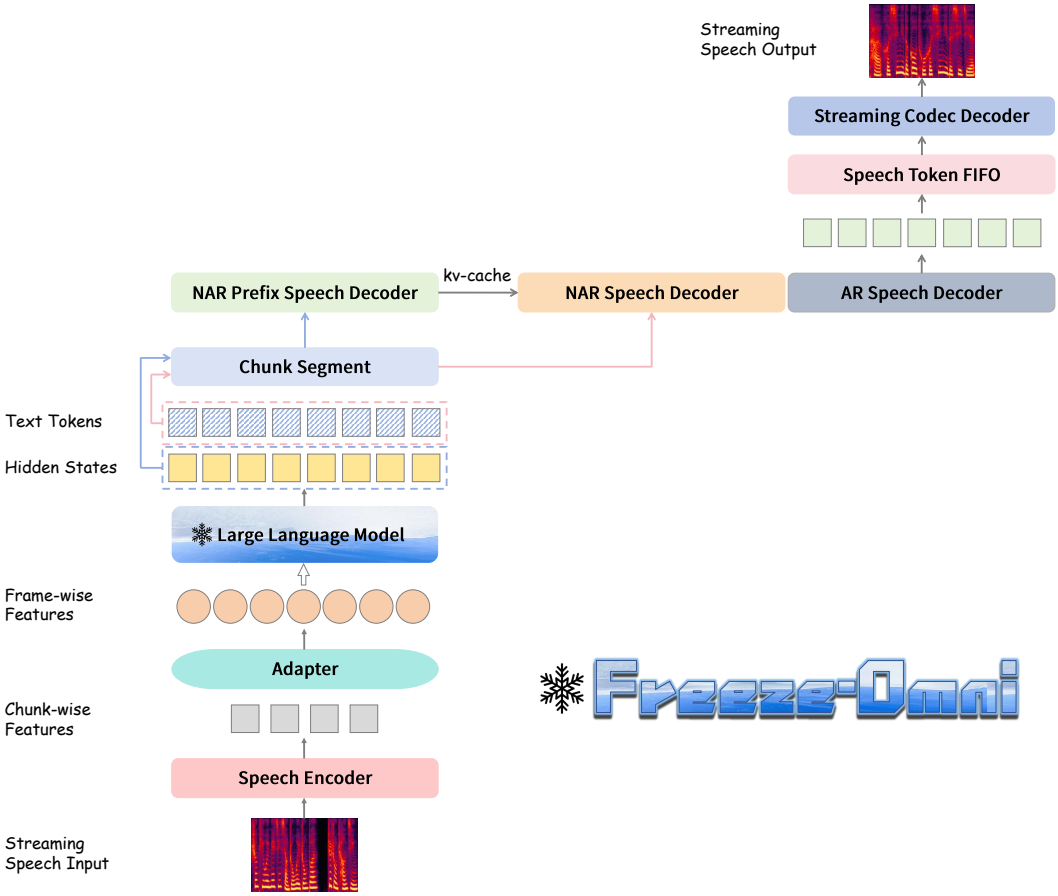

Figure 1: Overview of proposed Freeze-Omni. The streaming speech input forms chunk-wise features through the speech encoder, and then is connected to the LLM through the adapter. The LLM generates hidden states and text tokens, which are sent to the NAR prefix speech decoder and the NAR speech decoder in the form of chunks respectively after chunk segmentation. Finally, the AR speech decoder sends the generated tokens into the speech token FIFO and the streaming codec decoder generates streaming speech output from the FIFO according to a fixed speech token chunk size.

2.2 Modeling of speech input

2.2.1 Chunk-wise streaming speech encoder

To enable Freeze-Omni to support speech input and achieve a rapid and low-latency response to the input speech, it utilizes a chunk-wise streaming speech encoder to transform the input speech features into a high-dimensional representation. Then, an adapter module maps the high-dimensional representation into the embedding space of the backbone LLM. The speech encoder module here consists of several down-sampling convolution layers and several Transformer [22] blocks, while the adapter only comprises several down-sampling convolution layers. The reason for using down-sampling is to reduce the frame rate of the speech features, increase the speed of the LLM in the prefill stage, and decrease the latency.

2.2.2 Training strategy

A 3-stage training strategy shown in Fig. 2 is used for the speech encoder, enabling Freeze-Omni to acquire the ability to understand the streaming input speech while keeping the LLM frozen.

- The first stage shown in Fig. 2(a) is the same as the training process of a common speech recognition model. The input is speech features and the label is the transcript corresponding to the speech, CTC [13] is used as the loss function.
- In the second stage shown in Fig. 2(b), we use the speech encoder trained in the first stage as the initialization parameter and connect it with the LLM using an adapter. The output of the LLM still uses the transcript corresponding to the input speech as the label. Several trainable special tokens are added to the input part to guide the LLM in completing the training process at this stage. In this stage, except for the frozen LLM, the parameters of other networks are all trainable.
- In the last stage shown in Fig. 2(c), we first construct a dataset of multi-round questions and use the LLM backbone relied on in the training to generate multi-round answers. The dataset constructed in this way will be completely compatible with the LLM backbone. Subsequently, we use a multi-speaker TTS system to generate data in the speech modality for the questions part and add trainable prompt embedding before each question in the multi-round to guide the LLM to achieve the ability of speech input to text output. In this stage, the trainable special tokens in stage 2 will be dropped, only the prompt embedding part is trainable and they use the same parameters for each question, the speech encoder is frozen to maintain the acoustic robustness obtained from stage 2, and the LLM is also frozen to ensure that its intelligence is not affected.

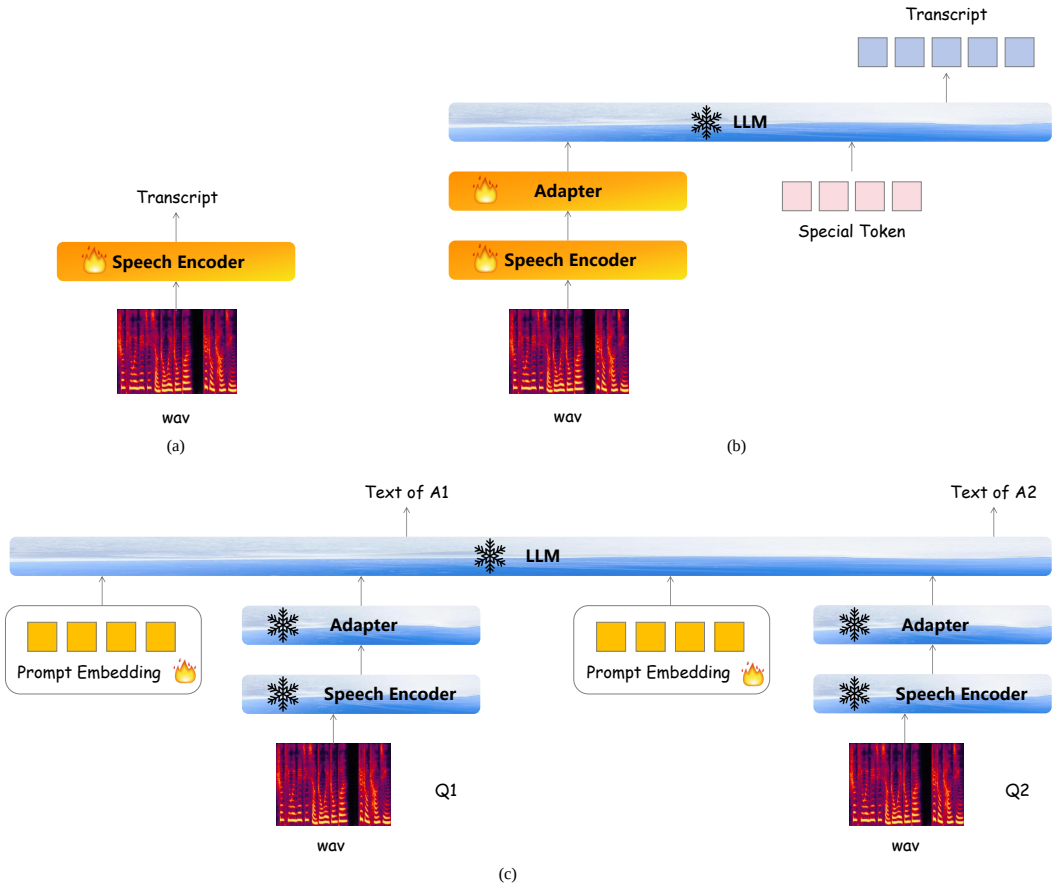

Figure 2: The 3-stage training method for modeling of speech input, the speech encoder in (c) is used in Freeze-Omni finally.

2.3 Modeling of speech output

2.3.1 Architecture

Inspired by VALL-E [5], Freeze-Omni uses a token-based speech decoder which contains NAR prefill and AR generate stage to achieve speech output capabilities. The speech decoder mainly consists of

the NAR decoder, the AR decoder, and the decoder of a codec model. Both the NAR decoder and AR decoder are built upon transformer blocks. The NAR decoder is used to model the semantic features from the output of LLM, and then the AR decoder generates speech tokens based on the output of the NAR decoder. Finally, a decoder of the codec model converts the speech tokens into a speech stream.

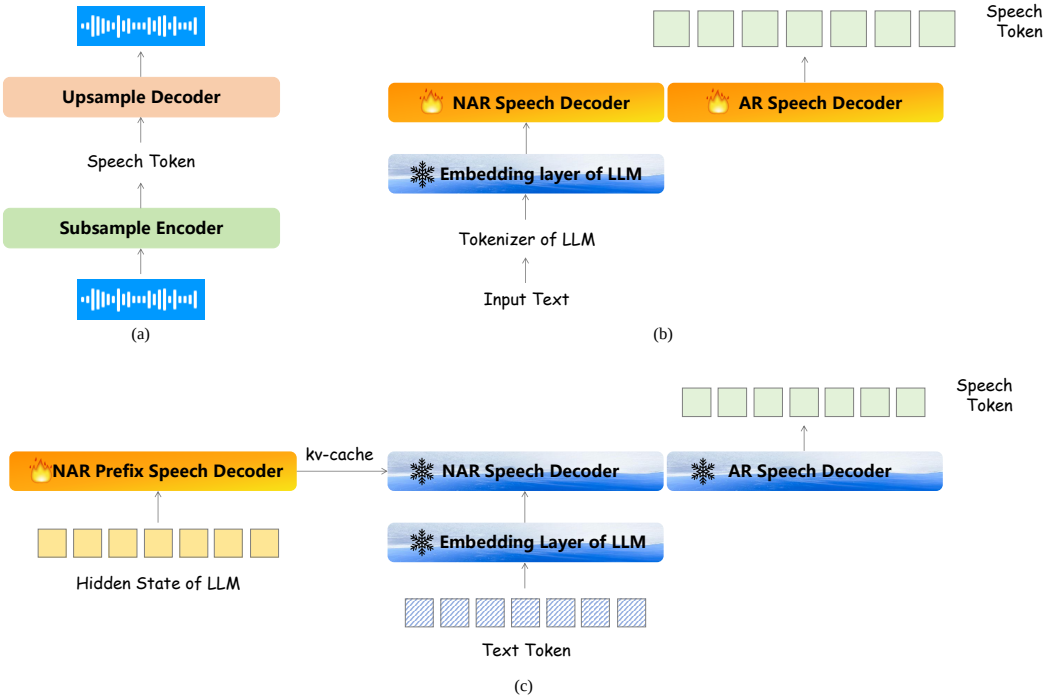

Figure 3: The 3-stage training method for modeling of speech output, the speech decoder in (c) is used in Freeze-Omni finally.

2.3.2 Training strategy

For the modeling of speech output, we still use a 3-stage training method as shown in Fig. 3, enabling Freeze-Omni to obtain the ability of generate speech from the output of LLM while keeping the LLM frozen.

- As shown in Fig. 3(a), we first train a single-codebook based codec model using only speech data. Since a single codebook is sufficient for extracting speech tokens from the speech signal of a limited number of speakers, using a single codebook here can reduce the complexity and latency of the system as much as possible.
- In the second stage shown in Fig. 3(b), we first construct a large amount of text-speech paired data and pass the text through the tokenizer of the backbone LLM to convert the text into text tokens. Then, we pass the text tokens through the embedding layer of the LLM to convert them into embedding vectors as semantic features and send them to the NAR speech decoder. The AR speech decoder predicts the output speech tokens in the form of teacher force. The labels here are extracted using the codec model trained in stage 1. The NAR and AR speech decoders use the same parameters, and the embedding layer of the LLM is frozen.
- In the last stage, we use the same multi-round questions and answers data set in stage 3 of Sec. 2.2.2 and use the text tokens and hidden state sequence generated by the backbone LLM. As shown in Fig. 3(c), an additional NAR prefix speech decoder is added to model the hidden state of the LLM and pass its output kv-cache to the NAR speech decoder. Then the text token will be fed to the NAR speech decoder trained in stage 2. The text token label for AR speech decoder is the speech data produced by the output text of LLM using a TTS system and converted into speech tokens by the codec model in stage 1. In this stage, the NAR prefix speech decoder uses different parameters

from the NAR and AR speech decoders, and only the parameters of the NAR prefix speech decoder are trainable while the parameters of other networks are frozen. Because the style of the text tokens produced by the LLM is different from that of the text in the large amount of text-speech paired data obtainable in stage 2, the significance of the third stage lies in closely coupling the speech decoder with the output of the LLM to reduce the occurrence of bad cases.

2.4 Design for duplex dialogue

After the above training process, Freeze-Omni has the ability of speech input to speech output. However, to better approximate the natural form of speech-to-speech dialogue, we use multi-task for chunk-level state prediction as shown in Fig 4. We first use an acoustic VAD[1] module to detect the starting point of the streaming speech. When the VAD is triggered, the speech stream will sent into Freeze-Omni chunk by chunk, and an additional classification layer will be added after the last layer of the LLM to predict different states. Three states are defined here, state 0 indicates that the current LLM can continue to receive speech, and state 1 or 2 indicates that the current chunk is the end of the speech. State 1 means that the user will interrupt the dialogue and the LLM will perform a new generate stage, and state 2 means that there is no need to interrupt the dialogue. Both states will stop sending speech streams to Freeze-Omni and reset the VAD module. The training process of this part is completed in stage 3 of Sec. 2.2.2, using a multi-task method to optimize the cross-entropy loss of both the state classification layer and the LLM. It should be noted that the state labels here are only valid on the last frame of each chunk.

Besides, we used a "model as a server" strategy to implement the speech-to-speech dialogue system. First, we started several models simultaneously and regarded them as a server. Then, when a user's VAD was triggered, the speech would be sent to the server in the form of chunks, and the server would be responsible for scheduling which idle model should respond to the current chunk. Since we separated all the kv-cache and CNN cache of the speech encoder and LLM during the inference process, the server only needs to save the inference cache for each user. In this way, any model in the server could respond to any chunk of any user, and there was no need to specify which model was used as a monitor or a generator.

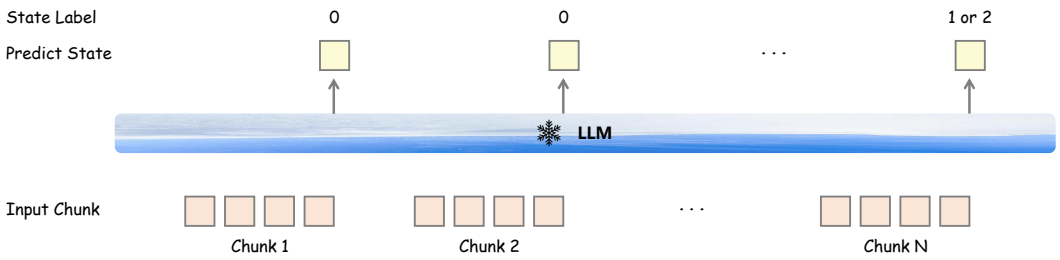

Figure 4: Method of chunk-level state prediction used in the prefill stage of the LLM. An additional classification layer is added to the output hidden state of the LLM corresponding to the last frame of each chunk output by the speech encoder to predict the state.

3 Experiments

3.1 Setups

3.1.1 Datasets

In this paper, we only randomly selected 60,000 multi-round Q&A data from *moss-003-sft-data* [2] and used backbone LLM to generate new answers to replace its original one. We used a zero-shot TTS system to synthesize its text into speech. For the modeling of speech input of Freeze-Omni, we used 110,000h internal speech-text paired ASR data including both Chinese and English in stage 1 and stage 2. In stage 3, we used the pairing of speech input and text output of the multi-round Q&A

[1] https://github.com/snakers4/silero-vad

[2] https://huggingface.co/datasets/fnlp/moss-003-sft-data

data mentioned above. For the modeling of the speech output of Freeze-Omni, we used about 3,000h of text-speech paired data generated by a zero-shot TTS system in stage 1 and stage 2. In stage 3, we used the pairing of text input and speech output of the multi-round Q&A data mentioned above.

3.1.2 Model configuration

LLM backend For experiments in this paper, we used Qwen2-7B-Instruct[3] as our backbone LLM. As an outstanding 7B-level public LLM, it is beneficial for us to verify our method. Besides, Freeze-Omni can use any LLM as a backbone in actuality because its training process does not update any parameters of the LLM.

Speech Encoder We used a multi-layer convolution with 4-times downsampling and 24 layers of transformers with a hidden size of 1024. The adapter consists of a multi-convolution layer with 2-times downsampling. The number of parameters for the speech encoder is approximately 350M, with an output frame rate of 12.5Hz. The input of the speech encoder is the mel-filter bank feature with a 25ms window size and 10ms shift.

Speech Decoder We used TiCodec[4] [20] as the codec model, and we customized the configuration so that the size of the codebook is 1024 with a single-codebook and the frequency of the speech token 40Hz. For the speech decoder part, both the NAR (Prefix) speech decoder and the AR speech decoder are 4-layer Llama decoder layers with a hidden size of 896. The number of parameters for the speech decoder is approximately 120M and the output sample rate of codec model is 24000Hz.

3.1.3 Training

In training processes we used the Adamw [16] optimizer with a warm-up learning rate scheduler, and different learning rates were used in different stages. The learning rates used in the three stages of the modeling of speech input are 2e-4, 1e-4, and 6e-4 respectively. The learning rates used in stage 2&3 of the modeling of speech output are both 5e-5 and the training hyper-parameters used in stage 1 are the same as that in TiCodec. All the experiments were completed on 8 GPUs.

3.2 Results on speech input

To measure the understanding ability of Freeze-Omni for input speech, as shown in Tab. 1, we verified the accuracy of ASR on different evaluation sets for the model in stage 2 of the modeling of speech input. Since the parameters of the speech encoder and adapter used in stage 3 are unchanged compared to those in stage 2, it can be considered that these results can represent the input speech understanding ability of Freeze-Omni. In the training of stage 2, we used a dynamic chunk training method [25] to enhance the robustness of the model so that different chunk sizes can be used in stage 3. From the results, it can be seen that in the case of dynamic chunk training, decoding with $chunk = \infty$ shows better performance compared to $chunk = 4$. If dynamic chunk training is not used but $chunk = 4$ decoding is used, better results can be obtained, but this also means that the chunk size cannot be changed in stage 3. In this paper, to pursue the best performance, all experiments are completed on the model with this configuration of the last row in Tab. 1.

3.3 Results on speech output

Because we investigated the speech-out performance of Freeze-Omni in a single-speaker case in this paper, we randomly selected 1,000 utterances of text tokens and hidden states output by the LLM as the input of the speech decoder and compared the ASR accuracy of the synthesized speech with the label text. As shown in Tab 2, the performance of the model in stage 2 of the modeling of speech output (Speech Decoder w/o Prefix) and the model in stage 3 (Speech Decoder) under different AR decoding parameters $top\text{-}k$ are presented respectively, and CER (%) is evaluated using *paraformer-zh*[5] [12]. From the results, it can be concluded that after introducing the hidden state of the LLM as the input of the NAR prefix speech decoder, the speech decoder can be more completely aligned with the LLM, reducing the occurrence of bad cases and get a lower CER (%). In addition, the increasing $top\text{-}k$ shows better robustness of the speech decoder with a prefix fine-tune.

[3] https://huggingface.co/Qwen/Qwen2-7B-Instruct

[4] https://github.com/y-ren16/TiCodec

[5] https://huggingface.co/funasr/paraformer-zh

In addition, the NAR and AR speech decoders need to model the LLM embedding outputs and speech tokens simultaneously, but the spaces represented by these two are different. Therefore, to verify whether the generation quality would be improved if the NAR speech decoder has additional parameters for modeling the outputs of the LLM embedding layer compared to the AR speech decoder, we added an extra pre-network between the NAR speech decoder and the LLM embedding layer. This pre-network consists of two Llama decoder layers with the same configuration of the NAR speech decoder. As shown in Tab 2, this method can significantly improve the speech quality generated by the speech decoder.

Table 1: The ASR performance of the model corresponding to stage 2 in the modeling of speech input, where {aishell-1 [4],test_net [27], test_meeting [27]} are Mandarin evaluation sets, measured in CER (%), while {dev-clean,dev-other,test-clean,test-other} [19] are English evaluation sets, measured in WER (%).

Model	aishell-1	test_net	test_meeting	dev-clean	dev-other	test-clean	test-other
Wav2vec2-base [2]	-	-	-	6.0	13.4	-	-
Mini-Omni2 [24]	-	-	-	4.8	9.8	4.7	9.4
Freeze-Omni							
+ $chunk = \infty$	2.15	8.57	10.09	3.29	7.4	3.24	7.68
+ $chunk = 4$	2.79	12.6	14.2	4.16	10.21	4.05	10.48
+ w/o dynamic	2.48	11.8	13.46	4.03	9.45	3.82	9.79

Table 2: The CER(%) of the speech decoder on 1,000 evaluation utterances under different $top\text{-}k$.

	$top\text{-}k$				
Method	1	2	3	4	5
Speech Decoder w/o Prefix	5.27	4.64	4.76	4.66	5.03
+ pre-network	3.11	2.75	2.77	2.84	2.94
Speech Decoder	3.9	3.65	3.53	3.62	3.71
+ pre-network	2.19	1.69	1.85	1.9	1.99

3.4 Results on spoken question answering

To demonstrate the intelligence of Freeze-Omni, we verified the accuracy of spoken question answering on three sets: LlaMA-Questions[6] [17], Web Questions[7] [3], and Trivia QA[8] [15]. Since Web Questions and Trivia QA only have text, we used the *edge-tts*[9] tool with voice at *en-US-BrianNeural* to synthesize them into spoken modality. Tab. 3 shows the accuracy of Freeze Omni and its used backbone LLM Qwen2-7B-Instruct on these three sets. From the results, it can observed that Freeze-Omni exhibits excellent performance compared to other models because the accuracy gap between it and the backbone LLM is smaller than that of Moshi, which also verifies that Freeze-Omni has the same level of intelligence in text and speech modalities.

3.5 Analysis on end-to-end latency

To verify the latency of Freeze-Omni for speech-to-speech dialogue, we defined two parts of latency, namely statistical latency and non-statistical latency. The statistical latency refers to the time from the LLM being interrupted to the first PCM chunk of speech generated. Specifically, it can be divided into four parts as shown in Fig 4, these results are based on a speech token chunk size of 40 and the use of text token chunk segmentation based on the sentence-split strategy. The non-statistical latency refers to the time from the real endpoint of speech to the LLM outputting the interrupt state. This part

[6] https://github.com/google-research-datasets/LLAMA1-Test-Set
[7] https://huggingface.co/datasets/Stanford/web_questions
[8] https://nlp.cs.washington.edu/triviaqa/
[9] https://github.com/rany2/edge-tts

needs to be measured manually and cannot be counted automatically. According to our case analysis conclusion, the non-statistical latency is about one to two speech encoder chunk sizes. According to the experiment configuration above, this time is about 160ms to 320ms. In summary, if we consider the influence of network latency (about 200 to 300ms), the average latency of Freeze-Omni used in real scenarios will be controlled at about 1.2 seconds.

Table 3: The accuracy (%) of different models in question answering on three sets. The models in the first four rows all use speech as input, while the models in the last two rows use text as input. The backbone LLM of Freeze-Omni is Qwen2-7B-Instruct, and the backbone LLM of Moshi is Helium. Both Freeze-Omni and Qwen2-7B-Instruct use greedy search in the generate stage with zero-shot, and the accuracy is calculated using the output text. Except for Freeze-Omni and Qwen2-7B-Instruct, previous evaluation results are derived from corresponding references.

Model	**Modality**	**Web Q.**	**LlaMA Q.**	**Audio Trivia QA**
SpeechGPT(7B) [28]	Audio&Text	6.5	21.6	14.8
Spectron(1B) [17]	Audio&Text	6.1	22.9	-
Moshi(7B) [7]	Audio&Text	26.6	62.3	22.8
GLM-4-Voice(9B) [26]	Audio&Text	32.2	64.7	39.1
Freeze-Omni(7B)	Audio&Text	**44.73**	**72**	**53.88**
Helium [7]	Text Only	32.3	75	56.4
Qwen2-7B-Instruct	Text Only	45.13	77.67	63.93

Table 4: Detailed information of statistical latency. Among them, 50% represents the median, and 90% represents the percentile at 90. The unit of the results in the table is (ms). All results are completed using pytorch with bfloat16 inference.

Latency description	**Avg.**	**50%**	**90%**
LLM interrupted $\rightarrow$ LLM generate first text token chunk	478	468	750
First text token chunk $\rightarrow$ Prefill of speech decoder	15	15	17
Prefill of speech decoder $\rightarrow$ Generate first speech token chunk	237	235	252
First speech token Chunk $\rightarrow$ Decode first PCM chunk	11	11	13
Total	745	753	1020

4 Conclusion and future work

In this paper, we proposed Freeze-Omni, a text-audio multimodal LLM capable of low-latency speech-to-speech dialogue, which does not need fine-tuning the LLM backbone, showing excellent performance in various tasks. In the future, to explore more speech dialogue capabilities, we plan to do the following updates:

- We will upgrade the speech encoder to a general audio encoder so that it can complete tasks such as emotion understanding and audio captioning.
- Under the condition of a frozen LLM, we will add more tasks to make the LLM complete more downstream tasks in speech dialogue like the state prediction ability.
- We plan to support multiple voices and instruct-following ability in the speech decoder part so that it can obtain more instruct information from the hidden state of the LLM and provide richer speaking styles.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33:12449–12460, 2020.

[3] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on Freebase from question-answer pairs. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1533–1544, Seattle, Washington, USA, October 2013. Association for Computational Linguistics.

[4] Hui Bu, Jiayu Du, Xingyu Na, Bengu Wu, and Hao Zheng. AISHELL-1: an open-source mandarin speech corpus and a speech recognition baseline. In *Conference of the Oriental Chapter of the International Coordinating Committee on Speech Databases and Speech I/O Systems and Assessment, O-COCOSDA 2017*, pages 1–5. IEEE, 2017.

[5] Sanyuan Chen, Shujie Liu, Long Zhou, Yanqing Liu, Xu Tan, Jinyu Li, Sheng Zhao, Yao Qian, and Furu Wei. Vall-e 2: Neural codec language models are human parity zero-shot text to speech synthesizers. *arXiv preprint arXiv:2406.05370*, 2024.

[6] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, et al. Qwen2-audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.

[7] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.

[8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[9] Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. Llama-omni: Seamless speech interaction with large language models. *arXiv preprint arXiv:2409.06666*, 2024.

[10] Luciano Floridi and Massimo Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020.

[11] Chaoyou Fu, Haojia Lin, Zuwei Long, Yunhang Shen, Meng Zhao, Yifan Zhang, Xiong Wang, Di Yin, Long Ma, Xiawu Zheng, et al. Vita: Towards open-source interactive omni multimodal llm. *arXiv preprint arXiv:2408.05211*, 2024.

[12] Zhifu Gao, Shiliang Zhang, Ian McLoughlin, and Zhijie Yan. Paraformer: Fast and accurate parallel Transformer for non-autoregressive end-to-end speech recognition. In *Proc. INTERSPEECH*, pages 5079–5083. ISCA, 2022.

[13] Alex Graves, Santiago Fernández, Faustino J. Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In William W. Cohen and Andrew W. Moore, editors, *International Conference on Machine Learning, ICML 2006*, volume 148 of *ACM International Conference Proceeding Series*, pages 369–376. ACM, 2006.

[14] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[15] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, Vancouver, Canada, July 2017. Association for Computational Linguistics.

[16] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *ArXiv*, abs/1711.05101, 2017.

[17] Eliya Nachmani, Alon Levkovitch, Roy Hirsch, Julian Salazar, Chulayuth Asawaroengchai, Soroosh Mariooryad, Ehud Rivlin, RJ Skerry-Ryan, and Michelle Tadmor Ramanovich. Spoken question answering and speech continuation using spectrogram-powered llm. *arXiv preprint arXiv:2305.15255*, 2023.

[18] OpenAI. `https://openai.com/index/hello-gpt-4o/`, 2024.

[19] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: An asr corpus based on public domain audio books. *IEEE*, 2015.

[20] Yong Ren, Tao Wang, Jiangyan Yi, Le Xu, Jianhua Tao, Chuyuan Zhang, and Junzuo Zhou. Fewer-token neural speech codec with time-invariant codes. *arXiv preprint arXiv:2310.00014*, 2023.

[21] Qwen Team. Qwen2.5: A party of foundation models, September 2024.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Annual Conference on Neural Information Processing Systems, NeurIPS 2017*, pages 5998–6008, 2017.

[23] Zhifei Xie and Changqiao Wu. Mini-omni: Language models can hear, talk while thinking in streaming. *arXiv preprint arXiv:2408.16725*, 2024.

[24] Zhifei Xie and Changqiao Wu. Mini-omni2: Towards open-source gpt-4o model with vision, speech and duplex. *arXiv preprint arXiv:2410.11190*, 2024.

[25] Zhuoyuan Yao, Di Wu, Xiong Wang, Binbin Zhang, Fan Yu, Chao Yang, Zhendong Peng, Xiaoyu Chen, Lei Xie, and Xin Lei. Wenet: Production oriented streaming and non-streaming end-to-end speech recognition toolkit. *arXiv preprint arXiv:2102.01547*, 2021.

[26] Aohan Zeng, Zhengxiao Du, Mingdao Liu, Lei Zhang, Shengmin Jiang, Yuxiao Dong, and Jie Tang. Scaling speech-text pre-training with synthetic interleaved data, 2024.

[27] Binbin Zhang, Hang Lv, Pengcheng Guo, Qijie Shao, Chao Yang, Lei Xie, Xin Xu, Hui Bu, Xiaoyu Chen, Chenchen Zeng, Di Wu, and Zhendong Peng. WENETSPEECH: A 10000+ hours multi-domain mandarin corpus for speech recognition. In *International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022*, pages 6182–6186. IEEE, 2022.

[28] Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu. Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities. *arXiv preprint arXiv:2305.11000*, 2023.